

Join the lecture online on your dashboard.

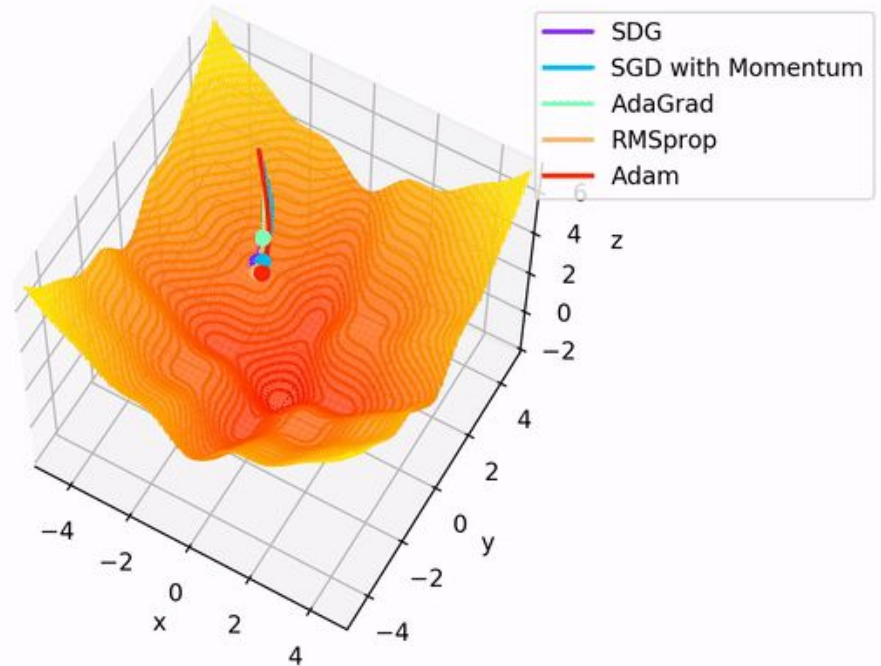
Backprop, Parameter initialization, Double Descent

Recap

So far we have seen:

1. How different gradient descent algorithms help us reach optimal model parameter values.
2. The impact that changing hyperparameters can have on the performance of these algorithms.

Optimizer Comparison



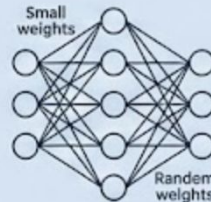
Agenda

Today we will take a look at:

1. **Motivation for Backpropagation.**
2. **Parameter Initialization**
3. **Double Descent**
4. **Understanding error tradeoffs and managing them**



1. Motivation for Backpropagation



2. Parameter Initialization



3. Double Descent



4. Understanding & Managing Errors

Problem Definition

Consider a network $f[x, \phi]$ with multivariate input x , parameters ϕ , and three hidden layers h_1, h_2 , and h_3 :

$$h_1 = a[\beta_0 + \Omega_0 x]$$

$$h_2 = a[\beta_1 + \Omega_1 h_1]$$

$$h_3 = a[\beta_2 + \Omega_2 h_2]$$

$$f[x, \phi] = \beta_3 + \Omega_3 h_3$$

function $a[\cdot]$ applies the activation function

$\phi = \{\beta_0, \Omega_0, \beta_1, \Omega_1, \beta_2, \Omega_2, \beta_3, \Omega_3\}$ consist of the bias vectors β_k and weight matrices Ω_k

Problem Definition

Loss function :

$$\ell_i = (f[x_i, \phi] - y_i)^2$$

Therefore total loss is :

$$L[\phi] = \sum_{i=1}^I \ell_i$$

Optimization Algorithm:

$$\text{SGD : } \phi_{t+1} \leftarrow \phi_t - \alpha \sum_{i \in \mathcal{B}_t} \frac{\partial \ell_i[\phi_t]}{\partial \phi}$$

To compute this update, we need to calculate the derivatives:

$$\frac{\partial \ell_i}{\partial \beta_k} \quad \text{and} \quad \frac{\partial \ell_i}{\partial \Omega_k}$$

for the parameters $\{\beta_k, \Omega_k\}$ at every layer $k \in \{0, 1, \dots, K\}$ and for each index i in the batch.

the backpropagation algorithm, which computes these derivatives efficiently.

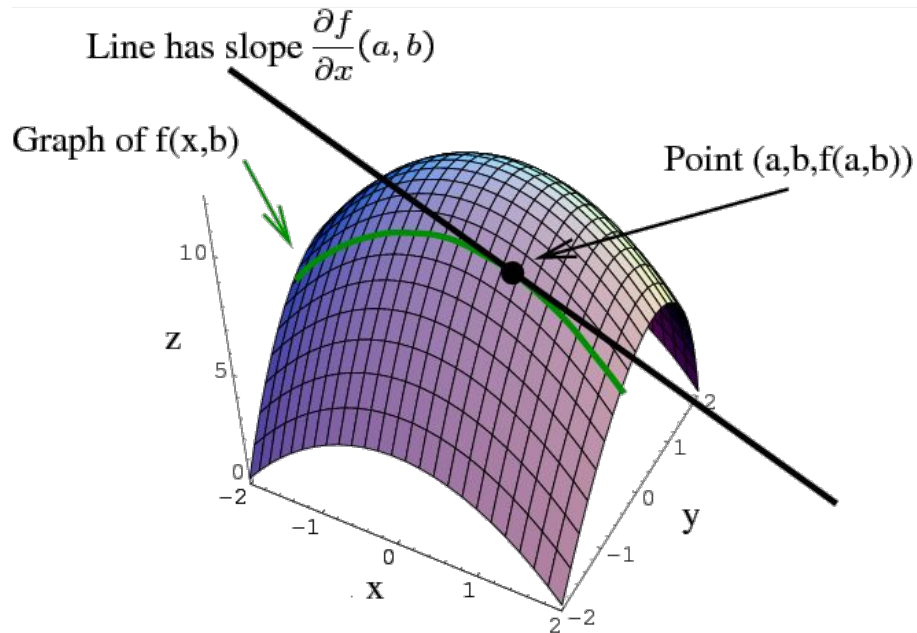
Computing Derivatives

Why do we even care about the derivatives of the loss ?

The derivatives of the loss tell us how the loss changes when we make a small change to the parameters.

Optimization algorithms use this information to find better parameters.

The backpropagation algorithm computes these derivatives.

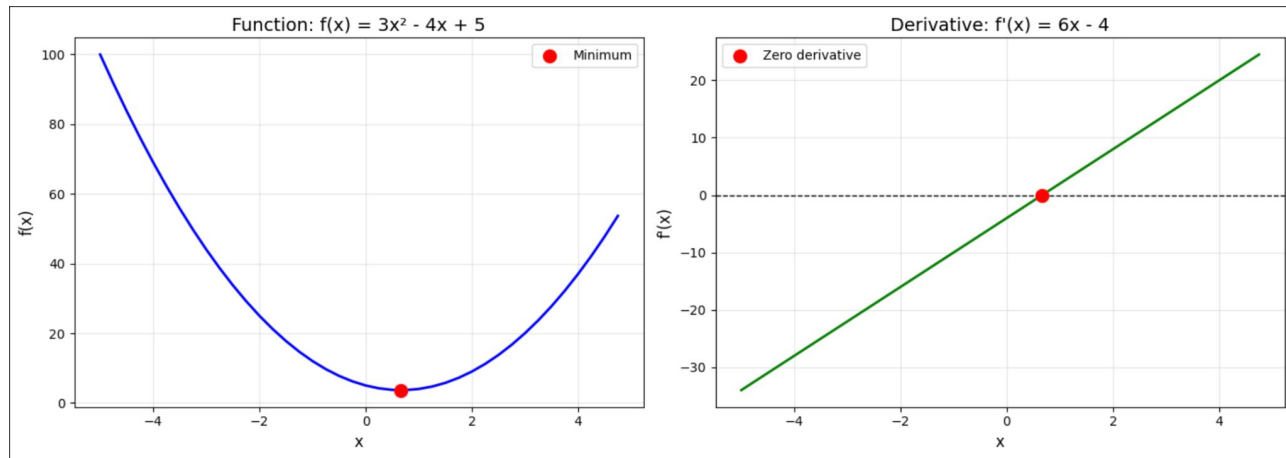


Intuition : Derivative

A derivative tells us:
"If I change the input slightly, how much does the output change?"

Notice that at $x = 2/3 = 0.6667$, the derivative is approximately 0 !

This is where the function has a minimum.



Intuition : Equation Graphs

Real neural networks have many variables.

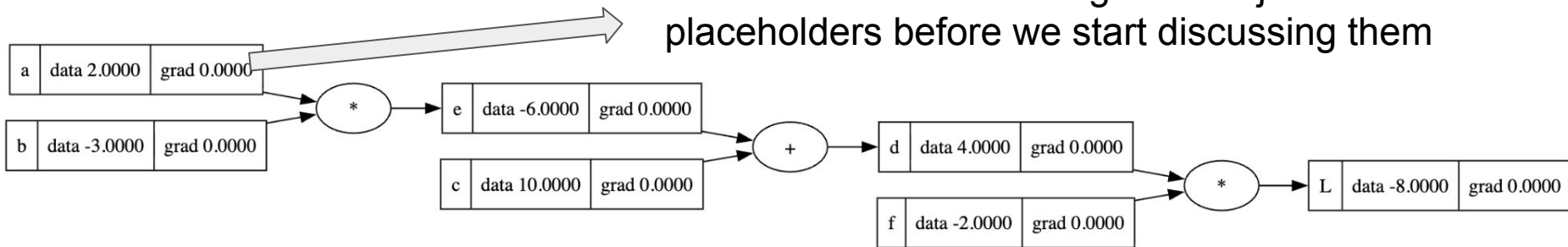
Let's consider a simple expression with multiple variables:

$$L = ((a * b) + c) * f$$

Here:

- a, b, c, f are inputs
- L is our final output (think: "Loss")

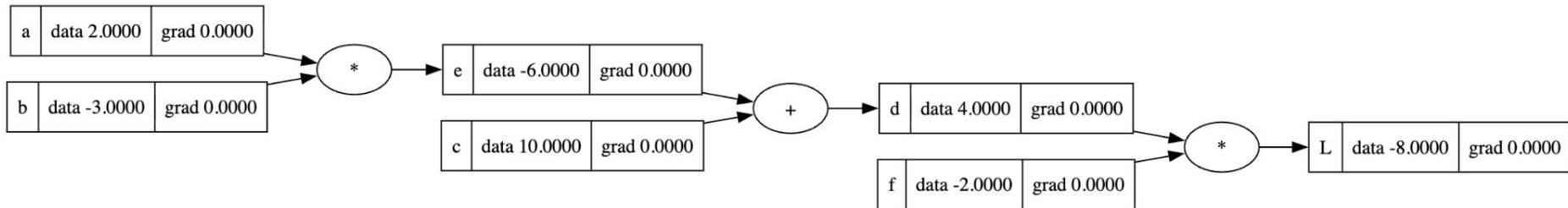
We want to know: How does each input affect the final output L?



Intuition : Derivatives + Equation graphs

Let's have a look at how to find these gradients via code:

[Collab Notebook](#)

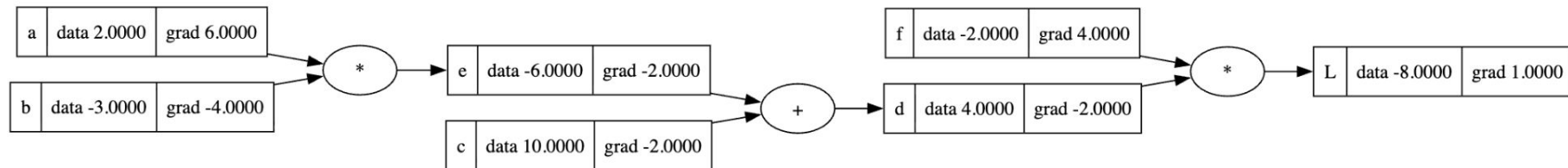


Summary : Derivatives + Equation graphs

What Do These Derivatives Mean?

- $\partial L / \partial a = 6.0$ If we increase a by 1, L increases by 6
- $\partial L / \partial b = -4.0$: If we increase b by 1, L decreases by 4 (because gradient is negative)
- $\partial L / \partial c = -2.0$: If we increase c by 1, L decreases by 2
- $\partial L / \partial f = 4.0$: If we increase f by 1, L increases by 4

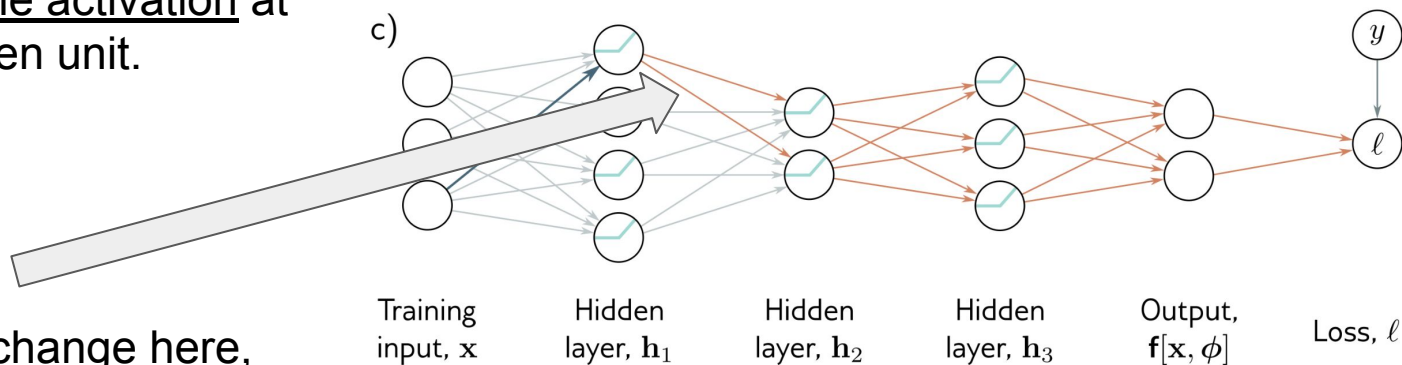
If we want to **decrease** L , we should:



Intuitive Observations

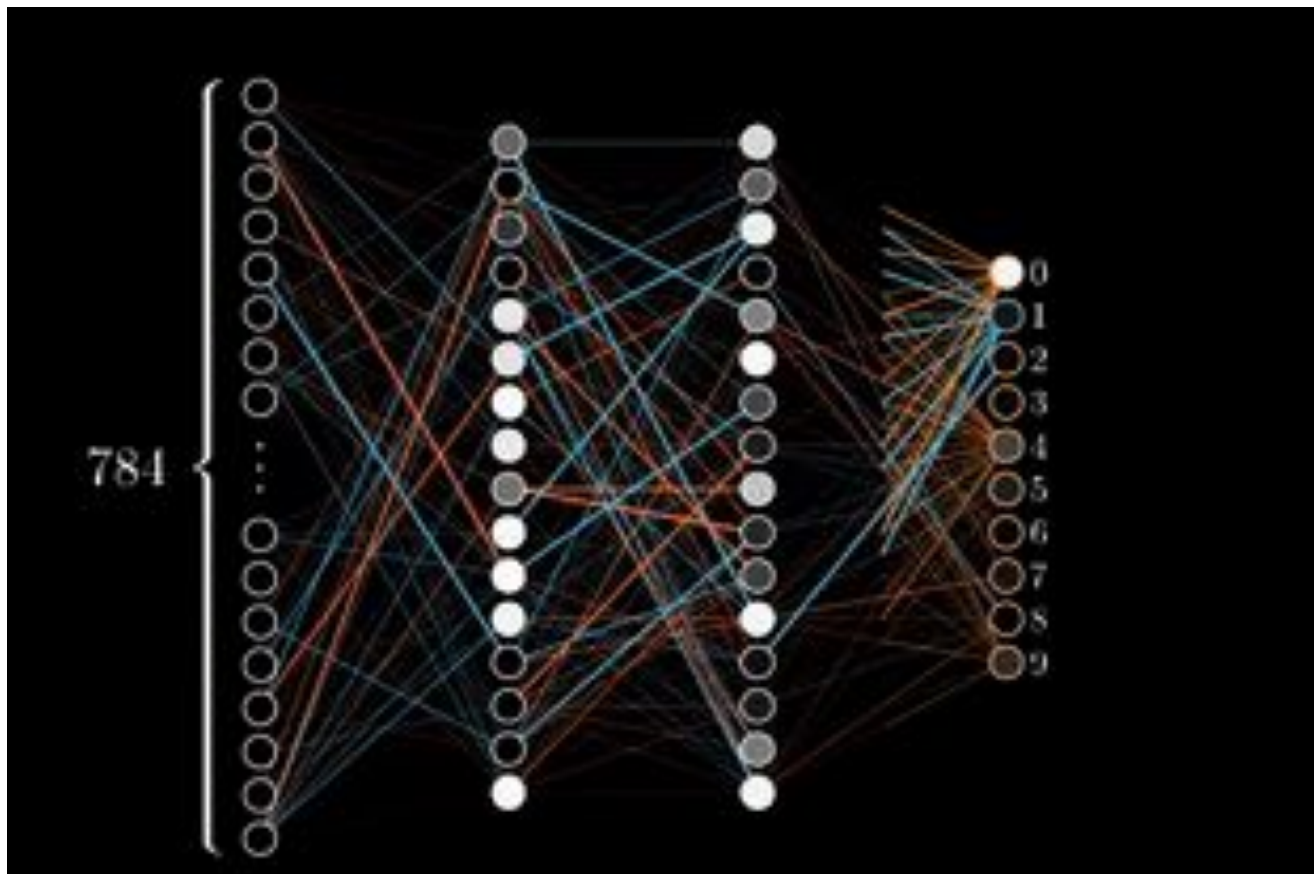
Observation 1: Each weight (element of Ω_k) multiplies the activation at a source hidden unit and adds the result to a destination hidden unit in the next layer. It follows that the effect of any small change to the weight is amplified or attenuated by the activation at the source hidden unit.

Observation 2: A small change in a bias or weight causes a ripple effect of changes through the subsequent network, until a change is made to the model output and, finally, the loss.



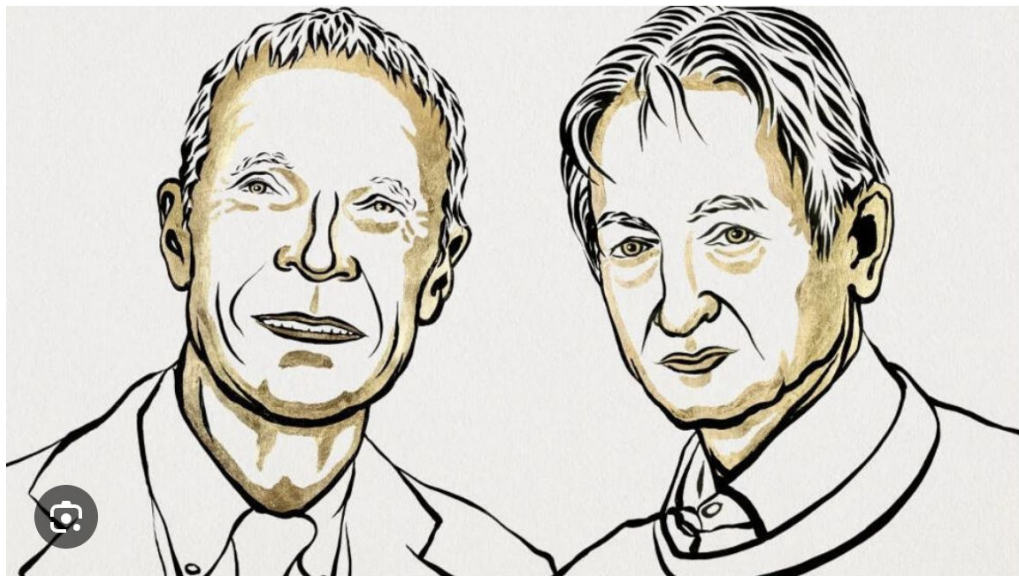
We made a small change here,
orange shows the impact

Backpropagation



Backpropagation

- Backpropagation is crucial for training neural networks.
- It is an efficient method for computing the gradient.
- The concept relies on the chain rule of calculus.
- Backprop updates weights to minimize the loss function.



From Backpropagation to the Nobel Prize: Hinton and Hopfield's Legacy in AI



Learning representations by back-propagating errors

David E. Rumelhart*, Geoffrey E. Hinton†
& Ronald J. Williams*

* Institute for Cognitive Science, C-015, University of California,
San Diego, La Jolla, California 92093, USA

† Department of Computer Science, Carnegie-Mellon University,
Pittsburgh, Philadelphia 15213, USA

We describe a new learning procedure, back-propagation, for networks of neurone-like units. The procedure repeatedly adjusts the weights of the connections in the network so as to minimize a measure of the difference between the actual output vector of the net and the desired output vector. As a result of the weight adjustments, internal 'hidden' units which are not part of the input or output come to represent important features of the task domain, and the regularities in the task are captured by the interactions of these units. The ability to create useful new features distinguishes back-propagation from earlier, simpler methods such as the perceptron-convergence procedure¹.



THE NOBEL PRIZE IN PHYSICS 2024



**John J. Hopfield and
Geoffrey E. Hinton**

for foundational discoveries and
inventions that enable machine learning
with artificial neural networks



Computational Graphs

These are **graphical** representations of **equations** !!

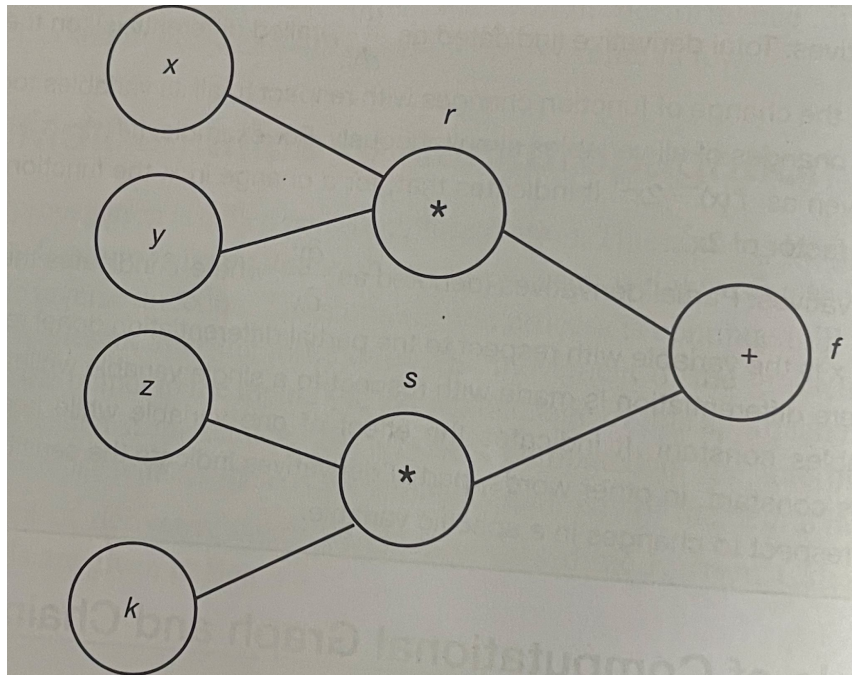
For an equation :

$$f = xy + zk$$

We can see that the variables are **x**, **y**, **z**, and **k**.

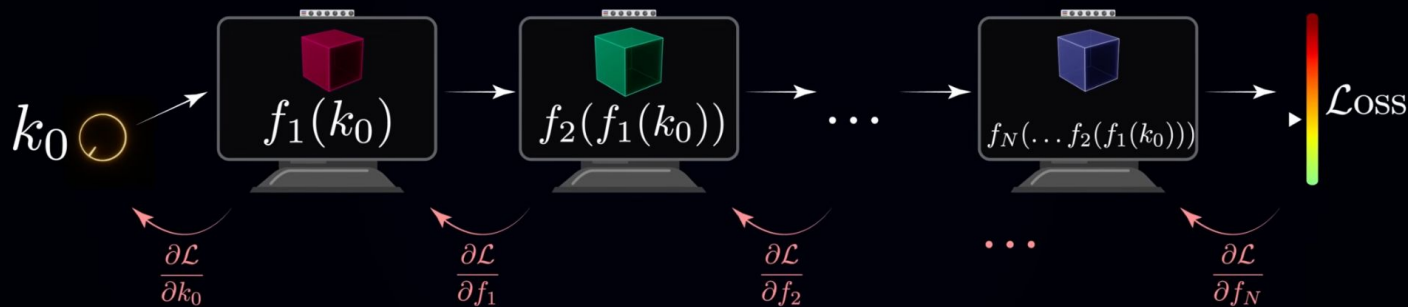
We setup **r** and **s** as intermediate variables, with the ***** operator.

Then we setup the final node **f** with the **+** operator



Big picture overview

1) For each of the parameters, write down its effect on the loss function in terms of a sequence of simple differentiable operations.



2) Sequentially apply the chain rule to compute the gradient of the loss with respect to each parameter

Solving for scalar function

Toy Example

$$f[x, \phi] = \beta_3 + \omega_3 \cdot \cos \left[\beta_2 + \omega_2 \cdot \exp \left[\beta_1 + \omega_1 \cdot \sin [\beta_0 + \omega_0 \cdot x] \right] \right]$$

We aim to compute the derivatives:

$$\frac{\partial \ell_i}{\partial \beta_0}, \quad \frac{\partial \ell_i}{\partial \omega_0}, \quad \frac{\partial \ell_i}{\partial \beta_1}, \quad \frac{\partial \ell_i}{\partial \omega_1}, \quad \frac{\partial \ell_i}{\partial \beta_2}, \quad \frac{\partial \ell_i}{\partial \omega_2}, \quad \frac{\partial \ell_i}{\partial \beta_3}, \quad \text{and} \quad \frac{\partial \ell_i}{\partial \omega_3}.$$

$$\begin{aligned} \frac{\partial \ell_i}{\partial \omega_0} = & -2 \left(\beta_3 + \omega_3 \cdot \cos \left[\beta_2 + \omega_2 \cdot \exp \left[\beta_1 + \omega_1 \cdot \sin [\beta_0 + \omega_0 \cdot x_i] \right] \right] - y_i \right) \\ & \cdot \omega_1 \omega_2 \omega_3 \cdot x_i \cdot \cos [\beta_0 + \omega_0 \cdot x_i] \cdot \exp \left[\beta_1 + \omega_1 \cdot \sin [\beta_0 + \omega_0 \cdot x_i] \right] \\ & \cdot \sin \left[\beta_2 + \omega_2 \cdot \exp \left[\beta_1 + \omega_1 \cdot \sin [\beta_0 + \omega_0 \cdot x_i] \right] \right]. \end{aligned}$$

The optimization challenge at scale

The Objective

Find parameters ϕ to minimize Loss $L[\phi]$.

Stochastic Gradient Descent (SGD) Update Rule

$$\phi_{t+1} \leftarrow \phi_t - \alpha \frac{\partial L}{\partial \phi}$$

Iteration t

Forward

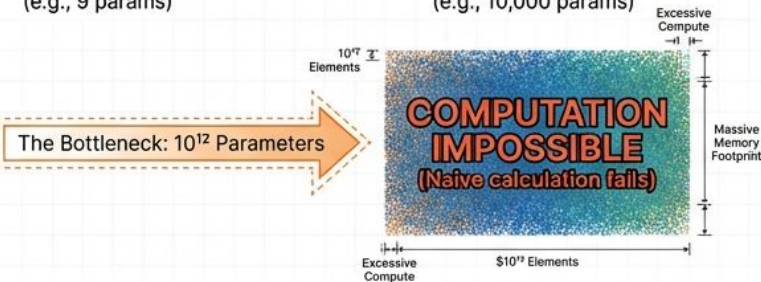
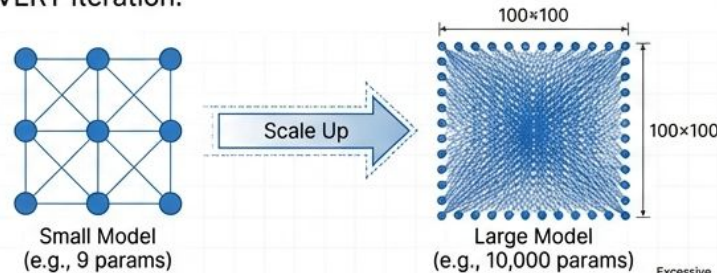
Loss

Iteration t+1

The Reality

Modern Deep Learning models contain $\sim 10^{12}$ parameters.

Requirement: Compute $\frac{\partial \ell_i}{\partial \phi}$ for EVERY parameter at EVERY iteration.



Forward pass: We treat the computation of the loss as a series of calculations:

$$f_0 = \beta_0 + \omega_0 \cdot x_i$$

$$h_1 = \sin[f_0]$$

$$f_1 = \beta_1 + \omega_1 \cdot h_1$$

$$h_2 = \exp[f_1]$$

$$f_2 = \beta_2 + \omega_2 \cdot h_2$$

$$h_3 = \cos[f_2]$$

$$f_3 = \beta_3 + \omega_3 \cdot h_3$$

$$\ell_i = (f_3 - y_i)^2.$$

We compute and store the values of the intermediate variables f_k and h_k (figure

Backward pass #1: We now compute the derivatives of ℓ_i with respect to these intermediate variables, but in reverse order:

$$\frac{\partial \ell_i}{\partial f_3}, \quad \frac{\partial \ell_i}{\partial h_3}, \quad \frac{\partial \ell_i}{\partial f_2}, \quad \frac{\partial \ell_i}{\partial h_2}, \quad \frac{\partial \ell_i}{\partial f_1}, \quad \frac{\partial \ell_i}{\partial h_1}, \quad \text{and} \quad \frac{\partial \ell_i}{\partial f_0}. \quad (7.10)$$

The first of these derivatives is straightforward:

$$\frac{\partial \ell_i}{\partial f_3} = 2(f_3 - y_i). \quad (7.11)$$

The next derivative can be calculated using the chain rule:

$$\frac{\partial \ell_i}{\partial h_3} = \frac{\partial f_3}{\partial h_3} \frac{\partial \ell_i}{\partial f_3}. \quad (7.12)$$

Extracting information in 1st pass

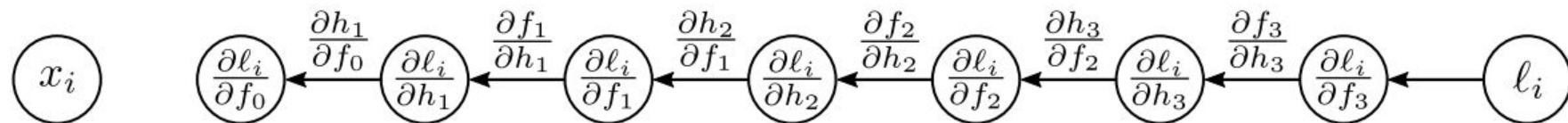


Figure 7.4 Backpropagation backward pass #1. We work backward from the end of the function computing the derivatives $\partial \ell_i / \partial f_k$ and $\partial \ell_i / \partial h_k$ of the loss with respect to the intermediate quantities. Each derivative is computed from the previous one by multiplying by terms of the form $\partial f_k / \partial h_k$ or $\partial h_k / \partial f_{k-1}$.

Backward pass #2: Finally, we consider how the loss ℓ_i changes when we change the parameters $\{\beta_k\}$ and $\{\omega_k\}$. Once more, we apply the chain rule (figure 7.5):

$$\begin{aligned}\frac{\partial \ell_i}{\partial \beta_k} &= \frac{\partial f_k}{\partial \beta_k} \frac{\partial \ell_i}{\partial f_k} \\ \frac{\partial \ell_i}{\partial \omega_k} &= \frac{\partial f_k}{\partial \omega_k} \frac{\partial \ell_i}{\partial f_k}.\end{aligned}\tag{7.14}$$

$$\frac{\partial f_k}{\partial \beta_k} = 1 \quad \text{and} \quad \frac{\partial f_k}{\partial \omega_k} = h_k.$$

Extracting more information in 2nd pass

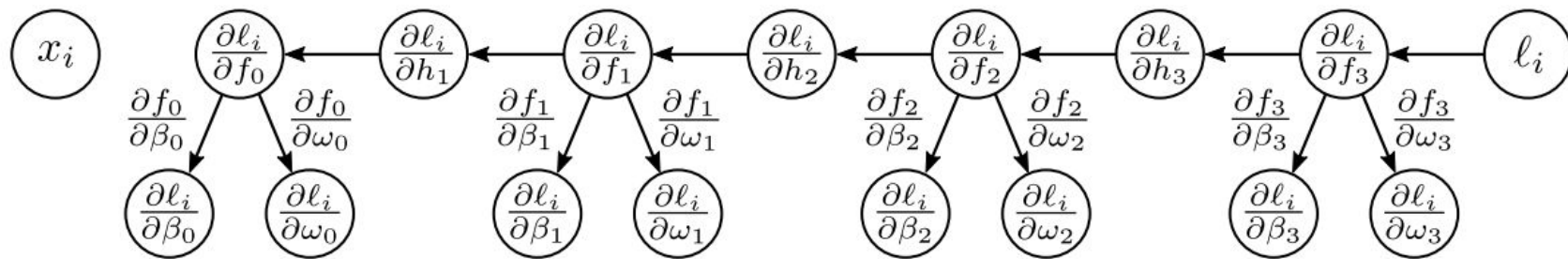
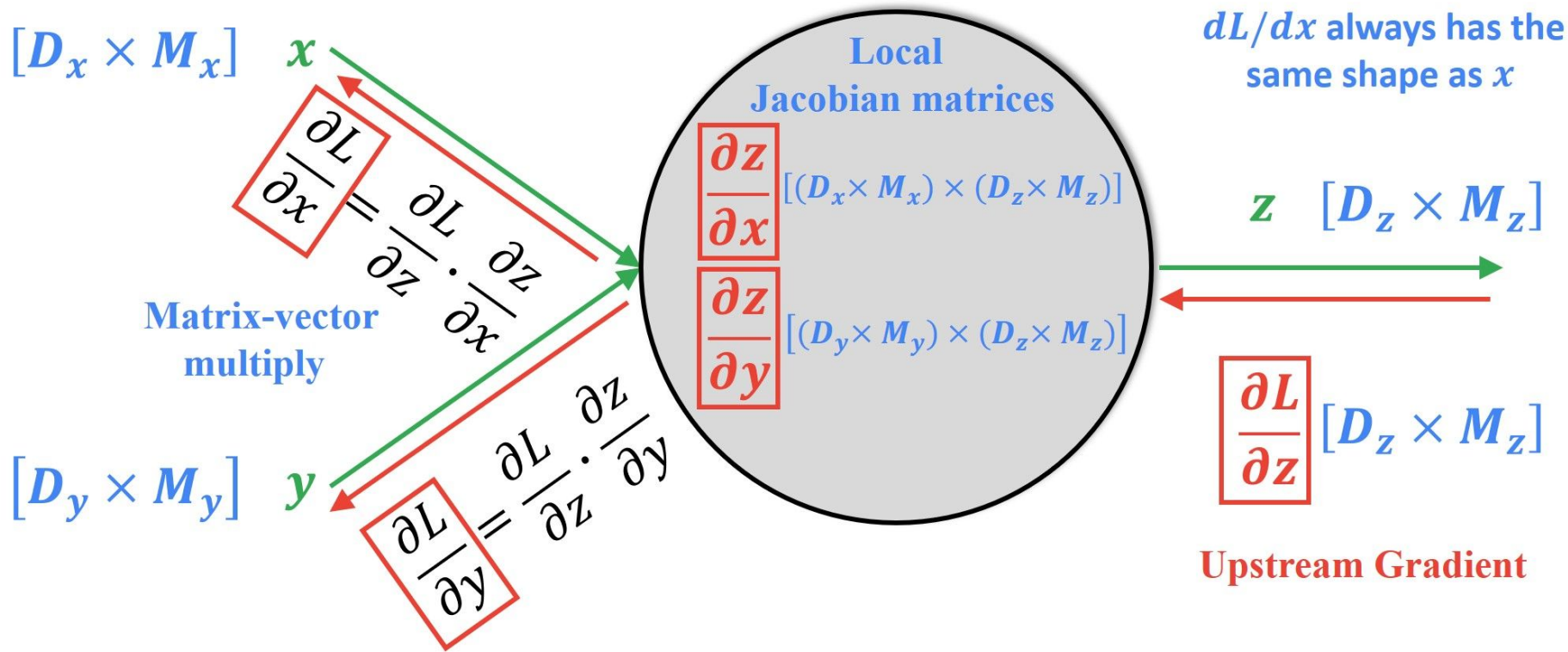


Figure 7.5 Backpropagation backward pass #2. Finally, we compute the derivatives $\partial \ell_i / \partial \beta_k$ and $\partial \ell_i / \partial \omega_k$. Each derivative is computed by multiplying the term $\partial \ell_i / \partial f_k$ by $\partial f_k / \partial \beta_k$ or $\partial f_k / \partial \omega_k$ as appropriate.

Generalizing for vector functions in Deep NNs



Matrix multiply: $y = xw$

$x: [N \times D]$

$w: [D \times M]$

$$\begin{bmatrix} 2 & 1 & -3 \\ -3 & 4 & 1 \end{bmatrix} \begin{bmatrix} 3 & 2 & 1 & -1 \\ 2 & 1 & 3 & 2 \\ 3 & 2 & 1 & -2 \end{bmatrix}$$

$$y_{n,m} = \sum_d x_{n,d} w_{d,m}$$

$y: [N \times M]$

$$\begin{bmatrix} -1 & -1 & 2 & 6 \\ 5 & 2 & 11 & 7 \end{bmatrix}$$

$dL/dx: [N \times D]$

$$\begin{bmatrix} 0 & 16 & -9 \\ -24 & 9 & 30 \end{bmatrix}$$

$dL/dy: [N \times M]$

$$\begin{bmatrix} 2 & 3 & -3 & 9 \\ -8 & 1 & 4 & 6 \end{bmatrix}$$

$$\frac{dL}{dx} = \left(\frac{dL}{dy} \right) \cdot w^T$$

$[N \times D]$

$[N \times M]$

$[M \times D]$

$dL/dx_{i,j}$

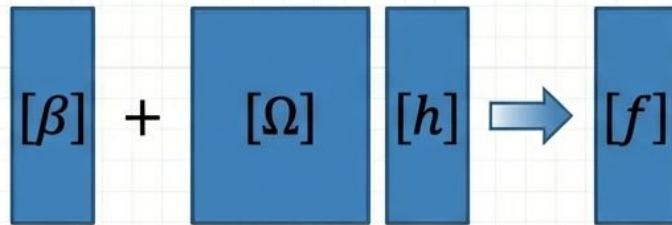
$$= (dy/dx_{i,j}) \cdot (dL/dy)$$

$$= (w_{j,:}) \cdot (dL/dy_{i,:})$$

The Backpropagation Algorithm (Matrix Formulation)

Forward Pass

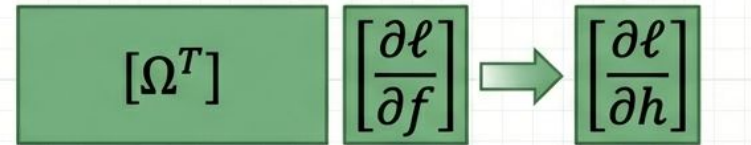
$$f_k = \beta_k + \Omega_k h_k$$



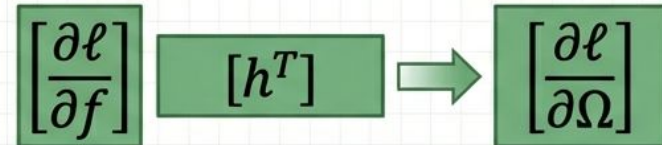
$$h_{k+1} = \text{ReLU}[f_k]$$

Backward Pass

$$\frac{\partial \ell}{\partial h_k} = \Omega_{k+1}^T \frac{\partial \ell}{\partial f_{k+1}}$$

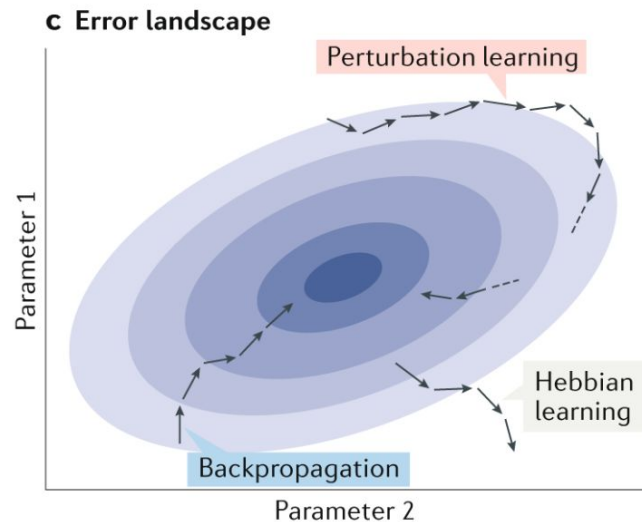
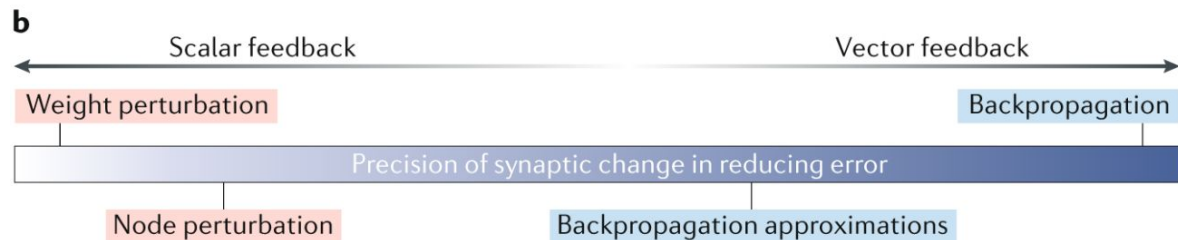
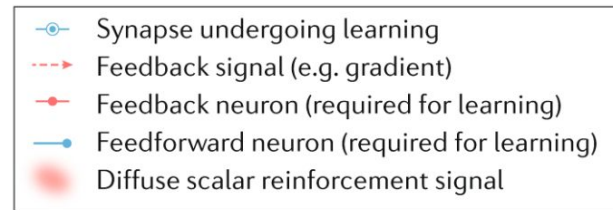
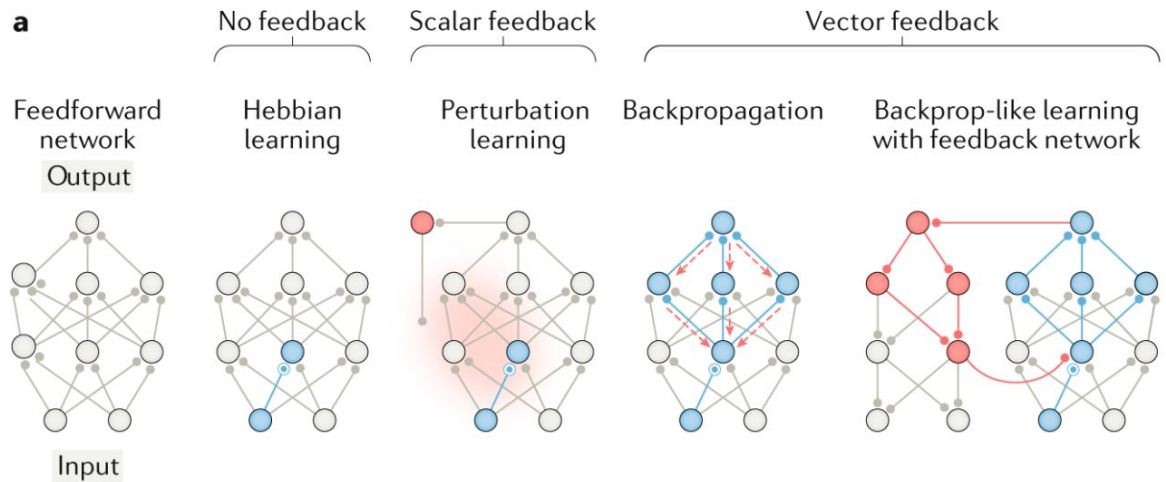


$$\frac{\partial \ell}{\partial \Omega_k} = \frac{\partial \ell}{\partial f_k} h_k^T$$



The matrix multiplication Ω^T is the most computationally expensive step.

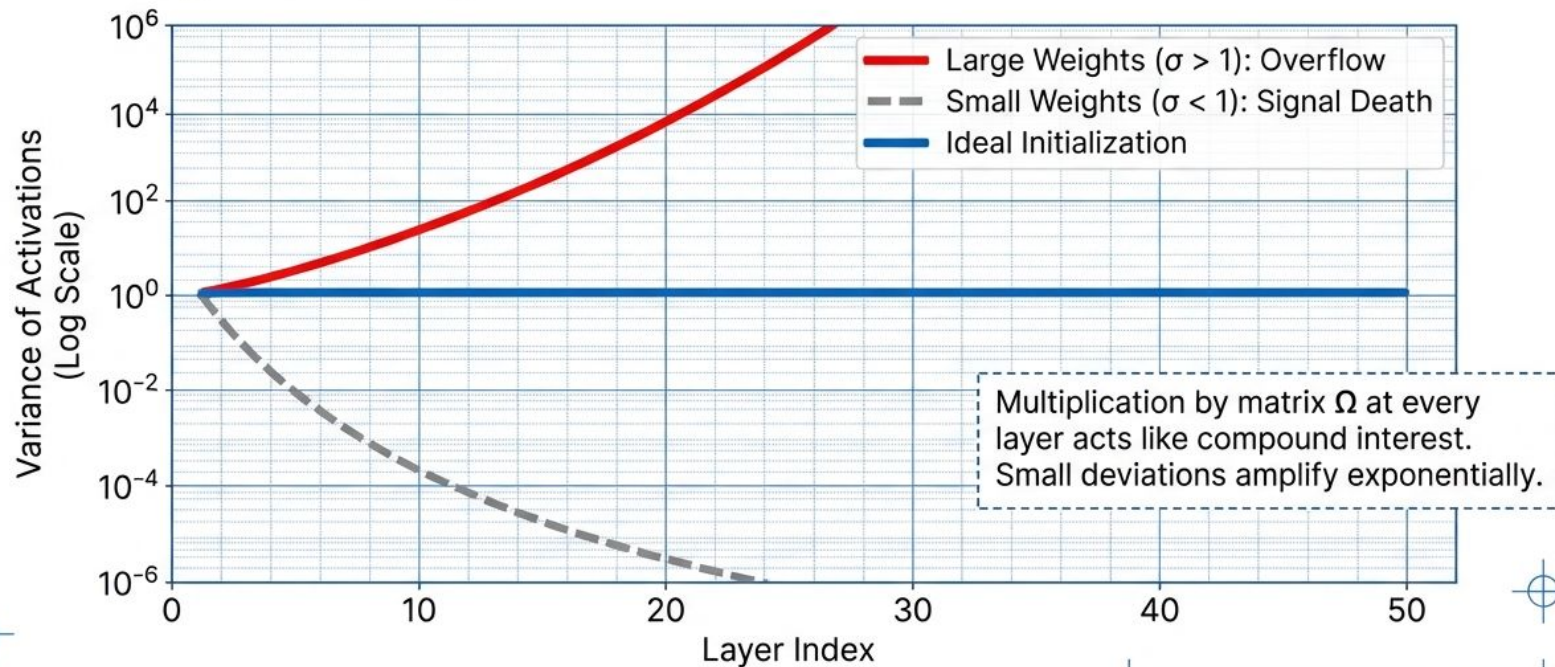
Other forms of learning



Parameter Initialization

The instability of deep architectures

Scenario: A 50-layer network with random weight initialization.
What happens to the signal?



Issues

- If the variance σ^2 is very small (e.g., 10^{-5}), then each element of $\beta_k + \Omega_k \mathbf{h}_k$ will be a weighted sum of $\mathbf{h}_k$ where the weights are very small; the result will likely have a smaller magnitude than the input. In addition, the ReLU function clips values less than zero, so the range of $\mathbf{h}_k$ will be half that of $\mathbf{f}_{k-1}$. Consequently, the magnitudes of the pre-activations at the hidden layers will get smaller and smaller as we progress through the network.
- If the variance σ^2 is very large (e.g., 10^5), then each element of $\beta_k + \Omega_k \mathbf{h}_k$ will be a weighted sum of $\mathbf{h}_k$ where the weights are very large; the result is likely to have a much larger magnitude than the input. The ReLU function halves the range of the inputs, but if σ^2 is large enough, the magnitudes of the pre-activations will still get larger as we progress through the network.

Mathematical analysis of variance propagation

Assumptions:

- Bias $\beta = 0$
- Weights $\Omega \sim N(0, \sigma^2)$
- Activations & Weights are independent

Step 1:

$$\text{Var}[\mathbf{f}'] = \text{Var}\left[\sum \Omega_i h_i\right] \rightarrow$$

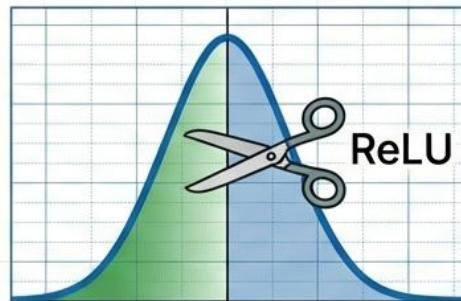
Step 2:

$$\text{Var}[\mathbf{f}'] = D_h \cdot \text{Var}[\Omega] \cdot \text{Var}[\mathbf{h}]$$

Sum of independent variances

Step 3:

$$\text{Var}[\mathbf{f}'] = D_h \cdot \sigma_{\Omega}^2 \cdot \text{Var}[\mathbf{h}]$$



ReLU sets half the values to 0.
Therefore, variance is halved.

$$\text{Var}[\mathbf{h}] = \frac{1}{2} \text{Var}[\mathbf{f}_{prev}]$$

$$\sigma_{f'}^2 = \frac{1}{2} D_h \sigma_{\Omega}^2 \sigma_f^2$$

Maths behind function variance

$$\begin{aligned}\sigma_{f'}^2 &= \mathbb{E}[f_i'^2] - \mathbb{E}[f_i']^2 \\ &= \mathbb{E} \left[\left(\beta_i + \sum_{j=1}^{D_h} \Omega_{ij} h_j \right)^2 \right] - 0 \\ &= \mathbb{E} \left[\left(\sum_{j=1}^{D_h} \Omega_{ij} h_j \right)^2 \right] \\ &= \sum_{j=1}^{D_h} \mathbb{E} [\Omega_{ij}^2] \mathbb{E} [h_j^2] \\ &= \sum_{j=1}^{D_h} \sigma_{\Omega}^2 \mathbb{E} [h_j^2] = \sigma_{\Omega}^2 \sum_{j=1}^{D_h} \mathbb{E} [h_j^2],\end{aligned}$$

$$\sigma_{f'_i}^2 = \sigma_{\Omega}^2 \sum_{j=1}^{D_h} \frac{\sigma_f^2}{2} = \frac{1}{2} D_h \sigma_{\Omega}^2 \sigma_f^2.$$

$$\sigma_{\Omega}^2 = \frac{2}{D_h},$$

This is known as **He initialization**.

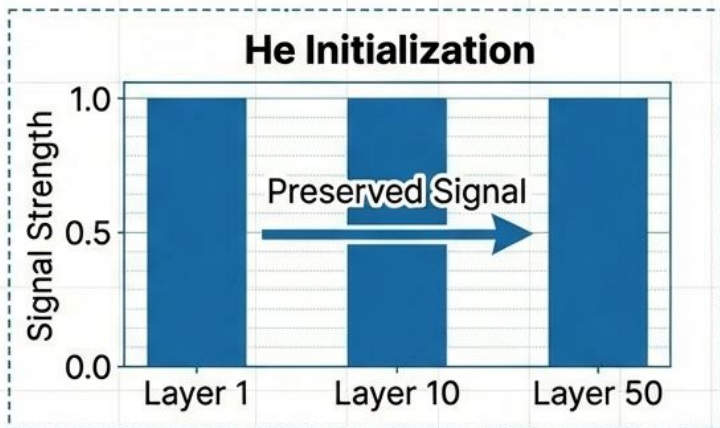
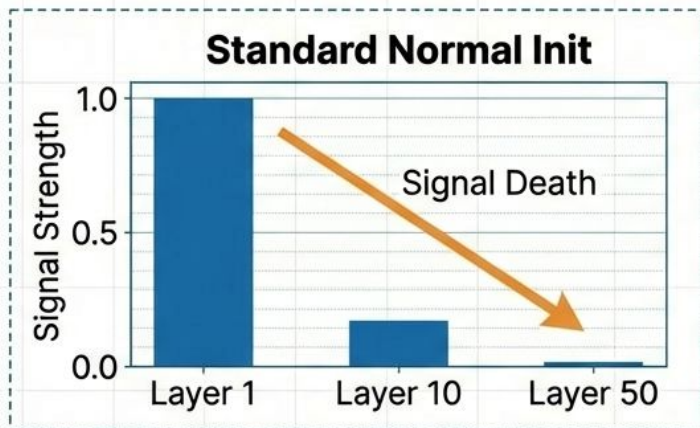
Stabilizing the engine: He Initialization

Goal: Preserve signal strength.
Input Variance = Output Variance.

$$\sigma_{f'}^2 = \sigma_f^2$$

The Fix

$$\frac{1}{2} D_h \sigma_{\Omega}^2 = 1 \Rightarrow \sigma_{\Omega}^2 = \frac{2}{D_h}$$

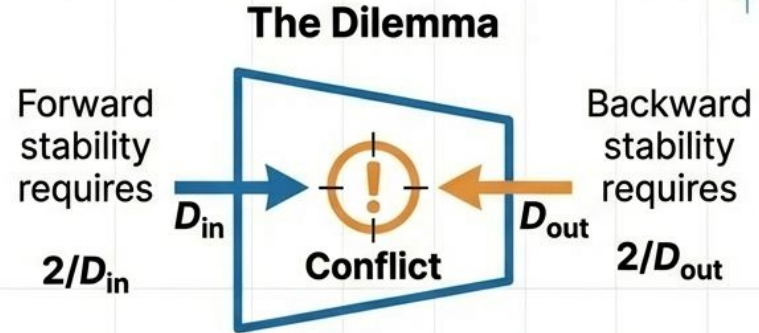


Also known as Kaiming Initialization. (Xavier/Glorot Init uses $1/D_h$, which is incorrect for ReLU).

Initialization for the backward pass

Constraint: During backprop, we multiply by Ω^T . To prevent vanishing gradients, we need:

$$\sigma_{\Omega^2}^2 = \frac{2}{D_{\text{out}}}$$



The Solution: The Harmonic Mean Compromise:

$$\sigma_{\Omega^2}^2 = \frac{4}{D_{\text{in}} + D_{\text{out}}}$$

Ensures reasonable stability for both activations (Forward) and gradients (Backward).

Implementation in PyTorch

```
import torch.nn as nn

# 1. Define Layer
layer = nn.Linear(D_in, D_out)

# 2. He Initialization (Kaiming Normal)
nn.init.kaiming_normal_(layer.weight)
layer.bias.data.fill_(0.0)

# 3. The Optimization Loop
optimizer.zero_grad()
loss = criterion(pred, y)

loss.backward() # <-- Autograd Engine Triggers Here
optimizer.step()
```

Implements $\sigma^2 = 2/D_{in}$

Traverses graph in reverse
to populate **.grad** attributes

```
import torch, torch.nn as nn
from torch.utils.data import TensorDataset, DataLoader
from torch.optim.lr_scheduler import StepLR

# define input size, hidden layer size, output size
D_i, D_k, D_o = 10, 40, 5
# create model with two hidden layers
model = nn.Sequential(
    nn.Linear(D_i, D_k),
    nn.ReLU(),
    nn.Linear(D_k, D_k),
    nn.ReLU(),
    nn.Linear(D_k, D_o))

# He initialization of weights
def weights_init(layer_in):
    if isinstance(layer_in, nn.Linear):
        nn.init.kaiming_normal_(layer_in.weight)
        layer_in.bias.data.fill_(0.0)
model.apply(weights_init)

# choose least squares loss function
criterion = nn.MSELoss()

# construct SGD optimizer and initialize learning rate and momentum
optimizer = torch.optim.SGD(model.parameters(), lr = 0.1, momentum=0.9)
# object that decreases learning rate by half every 10 epochs
scheduler = StepLR(optimizer, step_size=10, gamma=0.5)
```

```
# create 100 random data points and store in data loader class
x = torch.randn(100, D_i)
y = torch.randn(100, D_o)
data_loader = DataLoader(TensorDataset(x,y), batch_size=10, shuffle=True)

# loop over the dataset 100 times
for epoch in range(100):
    epoch_loss = 0.0
    # loop over batches
    for i, data in enumerate(data_loader):
        # retrieve inputs and labels for this batch
        x_batch, y_batch = data
        # zero the parameter gradients
        optimizer.zero_grad()
        # forward pass
        pred = model(x_batch)
        loss = criterion(pred, y_batch)
        # backward pass
        loss.backward()
        # SGD update
        optimizer.step()
        # update statistics
        epoch_loss += loss.item()
    # print error
    print(f'Epoch {epoch:5d}, loss {epoch_loss:.3f}')
    # tell scheduler to consider updating learning rate
    scheduler.step()
```

Double Descent

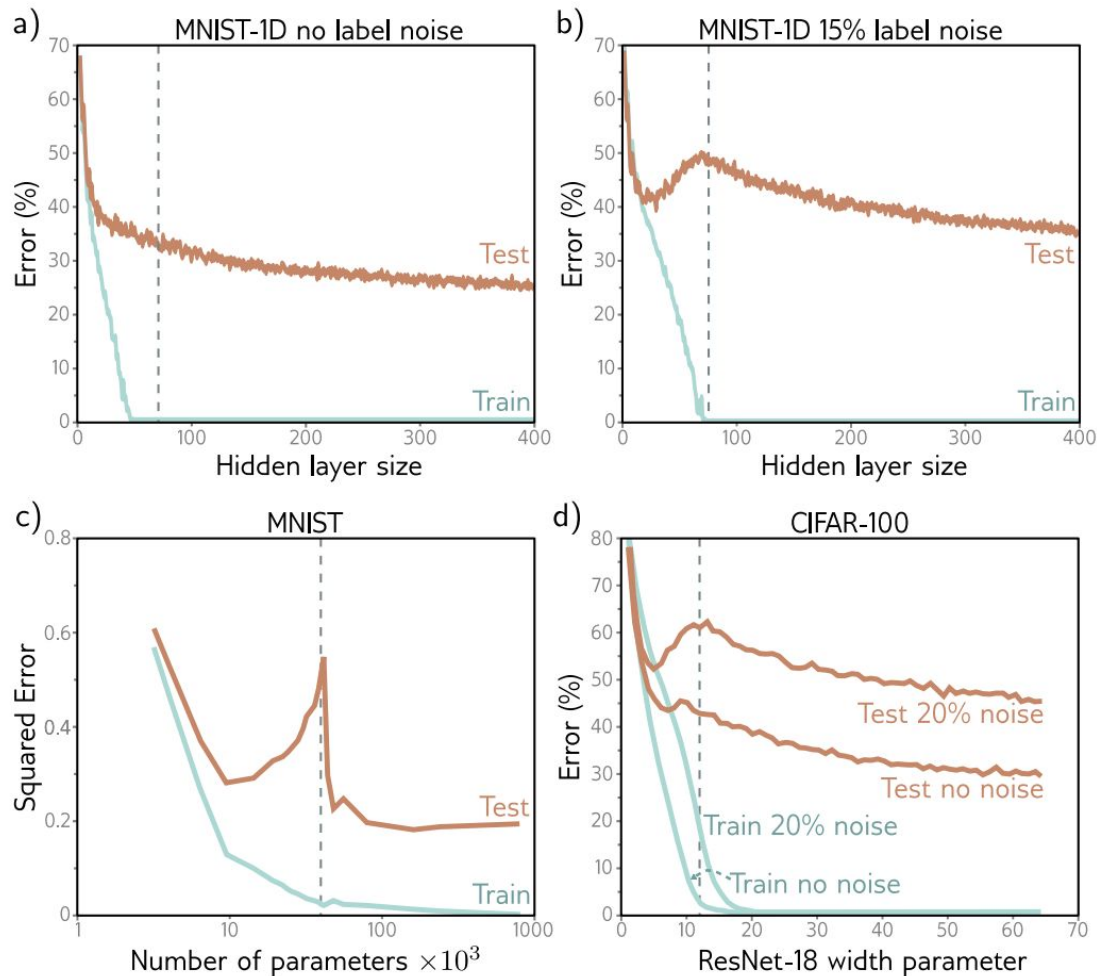


Figure 8.10 Double descent.

a) Training and test error on MNIST-1D for a two-hidden layer network as we increase the number of hidden units (and hence parameters) in each layer. The training error decreases to zero as the number of parameters approaches the number of training examples (vertical dashed line). The test error does not show the expected bias-variance trade-off but continues to decrease even after the model has memorized the dataset.

b) The same experiment is repeated with noisier training data. Again, the training error reduces to zero, although it now takes almost as many parameters as training points to memorize the dataset. The test error shows the predicted bias/variance trade-off; it decreases as the capacity increases but then increases again as we near the point where the training data is exactly memorized. However, it subsequently decreases again and ultimately reaches a better performance level. This is known as double descent. Depending on the loss function, the model, and the amount of noise in the data, the double descent pattern can be seen to a greater or lesser degree across many datasets.

c) Results on MNIST (without label noise) with shallow neural network from Belkin et al. (2019).

d) Results on CIFAR-100

Explanation

The discovery of double descent is recent, unexpected, and somewhat puzzling.

It results from an interaction of two phenomena.

First, the test performance becomes temporarily worse when the model has just enough capacity to memorize the data.

Second, the test performance continues to improve with capacity even when this exceeds the point where the training data are all classified correctly.

Explanation

The first phenomenon is exactly as predicted by the bias-variance trade-off.

The second phenomenon is more confusing; it's unclear why performance should be better in the over-parameterized regime, given that there are now not even enough training data points to constrain the model parameters uniquely.

To understand why performance continues to improve as we add more parameters, note that once the model has enough capacity to drive the training loss to near zero, the model fits the training data almost perfectly.

This implies that further capacity cannot help the model fit the training data any better; any change must occur between the training points.

The tendency of a model to prioritize one solution over another between data points is known as its inductive bias.

Understanding Double Descent

Experimental Setup

- Data: MNIST-1D (10k train / 5k test).
- Model: 2-layer neural network with increasing capacity (hidden units).
- Optimization: Adam optimizer, full batch, 4,000 steps.
- Variable: Comparing clean labels vs. 15% randomized labels.

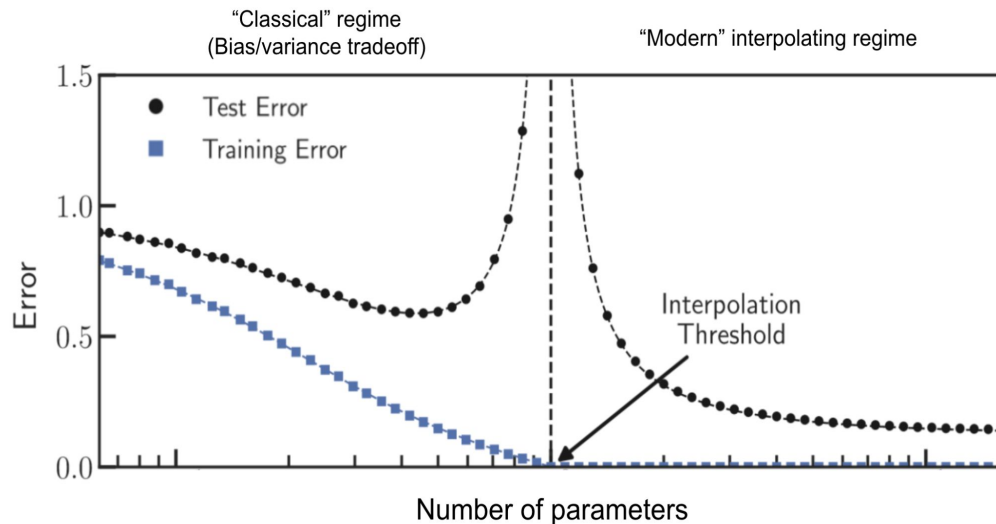
Key Observations

- Training Error: Always decreases to zero as capacity increases
- Test Error: Does not follow the simple U-shaped Bias-Variance curve. Instead, it exhibits Double Descent:
 - Error drops, then rises near the "interpolation threshold"
 - Unexpectedly, error drops again in the over-parameterized regime, often reaching a new global minimum.

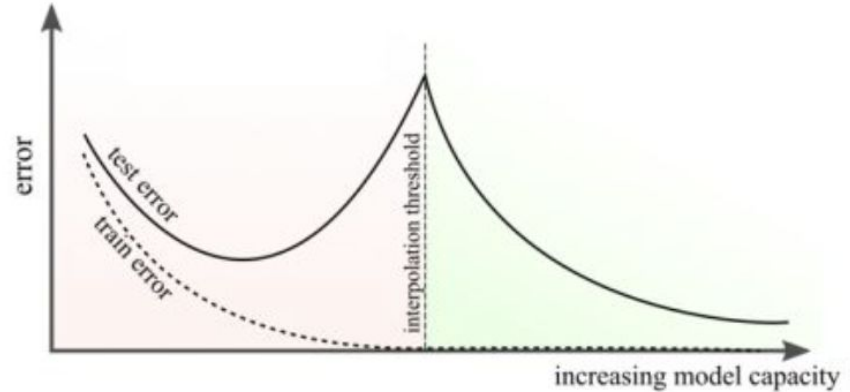
Understanding Double Descent : Regimes

The Three Regimes

- Under-parameterized (Classical): Error decreases as model capacity improves.
- Critical: Model has just enough parameters to "memorize" noise; test error peaks.
- Over-parameterized (Modern): Extra parameters provide a smoothing effect, leading to better generalization.



Relationship between Dunning Kruger effect and Double Descent



Tradeoffs while training..

Measuring Error

Now that we have seen how to shift any function in any direction using backpropagation, our neural networks will work well on training data.

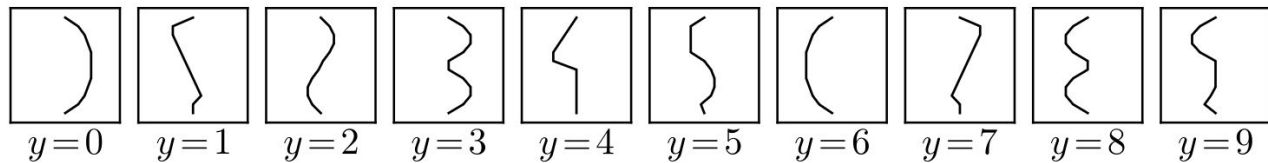
In case our models do not perform well on testing data we arbitrage with the following root causes:

1. The inherent uncertainty in the task
2. The amount of training data
3. The choice of model

Sample Dataset : MNIST-1D

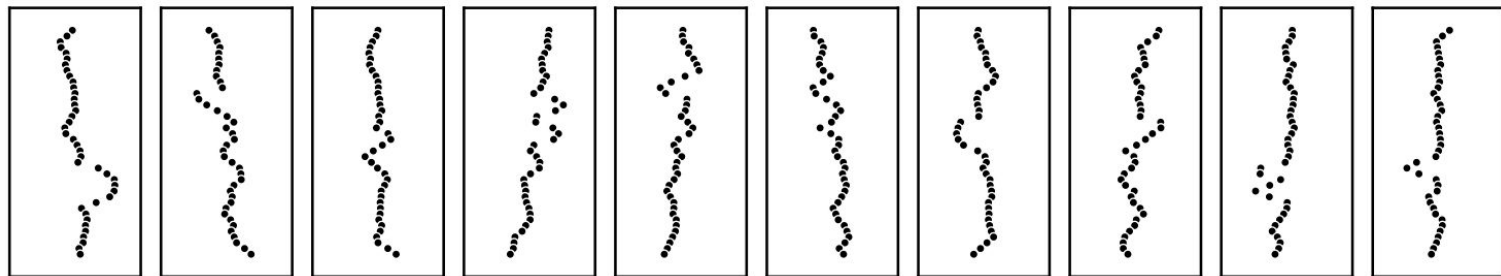
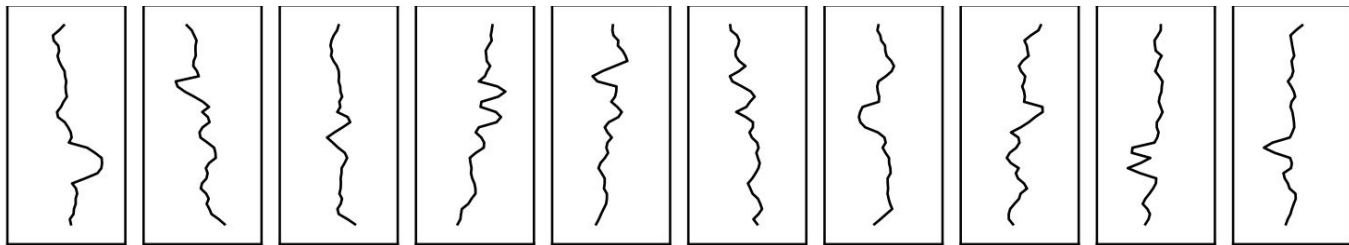
1. The Dataset
 - a. Task: 10-class digit classification $y \in [0,9]$,
 - b. Format: 40-dimensional signals derived from 1D templates + random transformation + noise.
 - c. Size: 4,000 total training examples (balanced at ~400 per class).
2. Model Architecture & Training
 - a. Network: 2 hidden layers (100 units each) with Softmax output.
 - b. Loss Function: Multiclass Cross-Entropy.
 - c. Optimization: SGD with 0.1 learning rate and 100 batch size.
 - d. Duration: 6,000 steps (150 epochs).

Sample Dataset : MNIST-1D



Templates for 10
classes $y \in \{0, \dots, 9\}$,

Training examples x
are created by
randomly transforming
+ adding noise to
templates



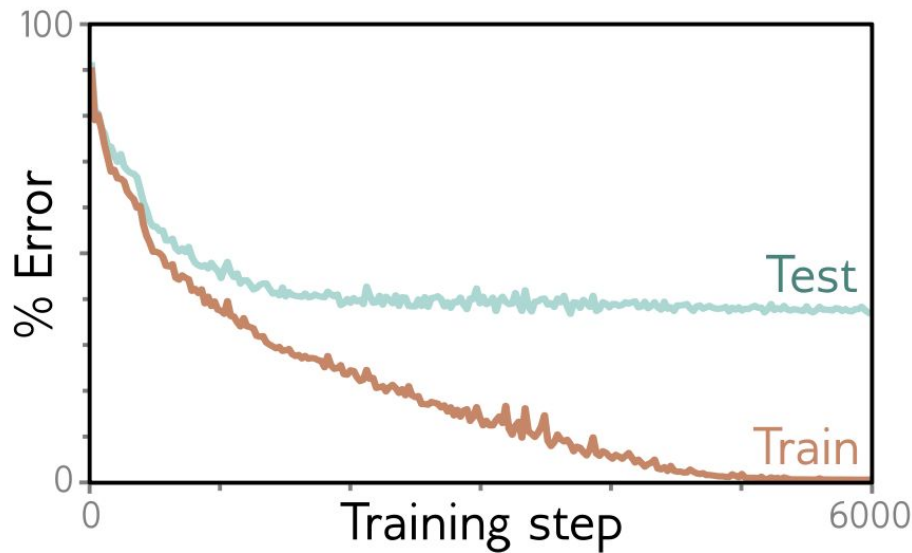
The horizontal
offset of the
transformed
template,
sampled at 40
positions

How do you think this model performs over the epochs ?

In terms of % error

- On the training set : Model learns test set perfectly
- On the test set : Model does not generalize well to new unseen samples.

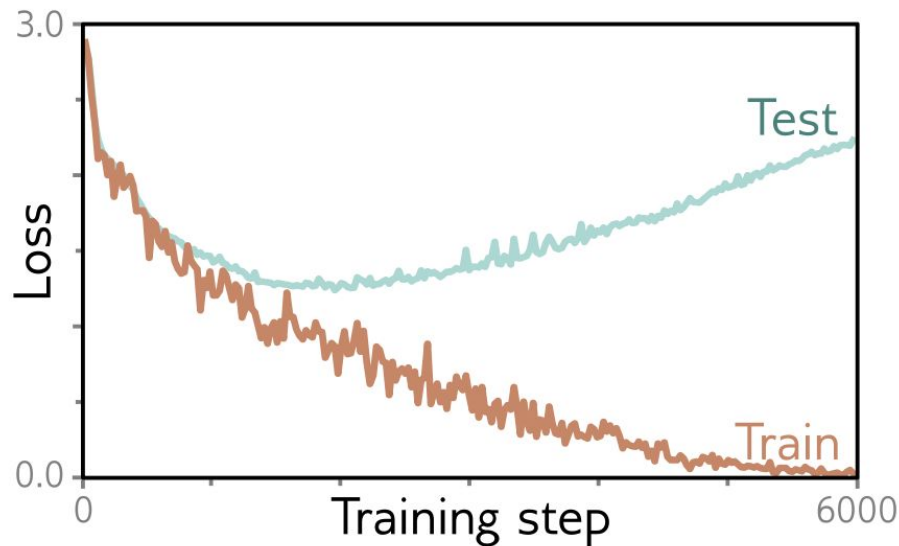
What about loss vs Training step graph ?



How do you think this model performs over the epochs ?

In terms of Loss

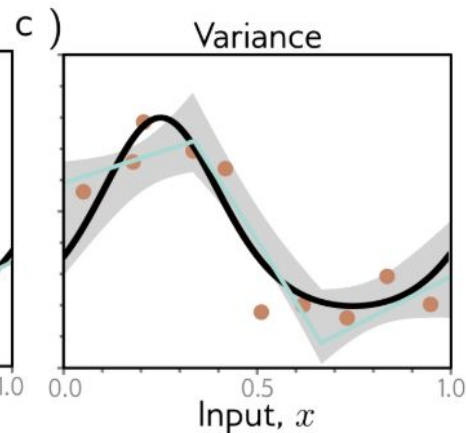
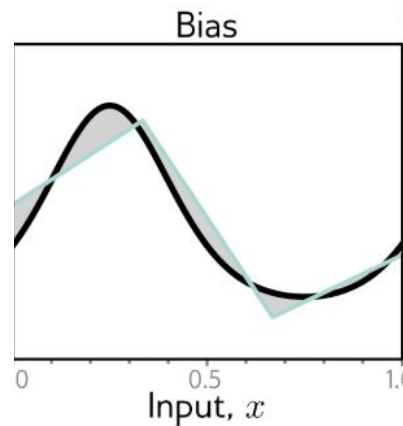
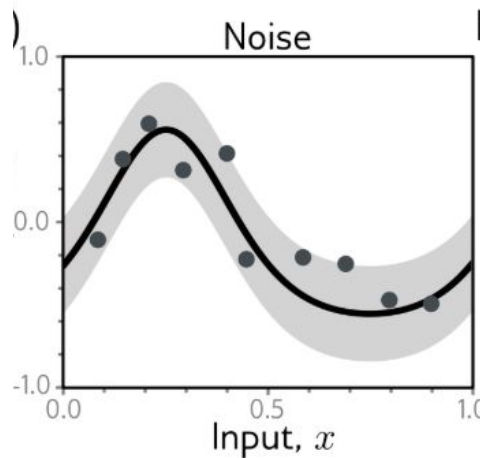
- On the training set : Model minimizes loss on training set
- On the test set : The model makes the same mistakes but with increasing confidence. This decreases the probability of the correct answers and thus increases the negative log-likelihood.



Understanding the sources of errors

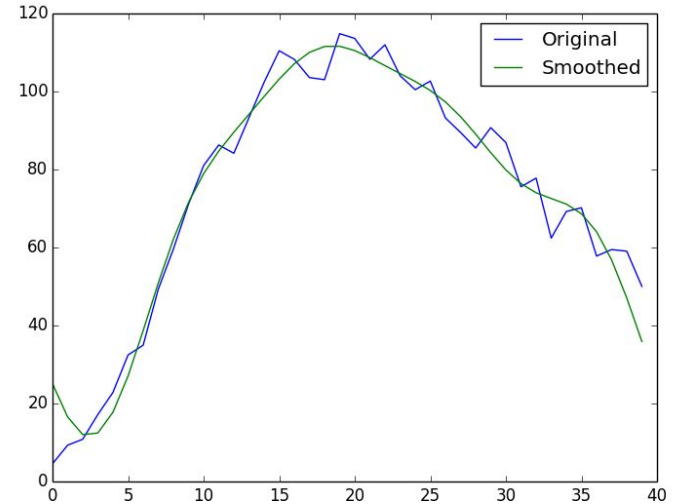
Sources of error:

1. Noise
2. Bias
3. Variance



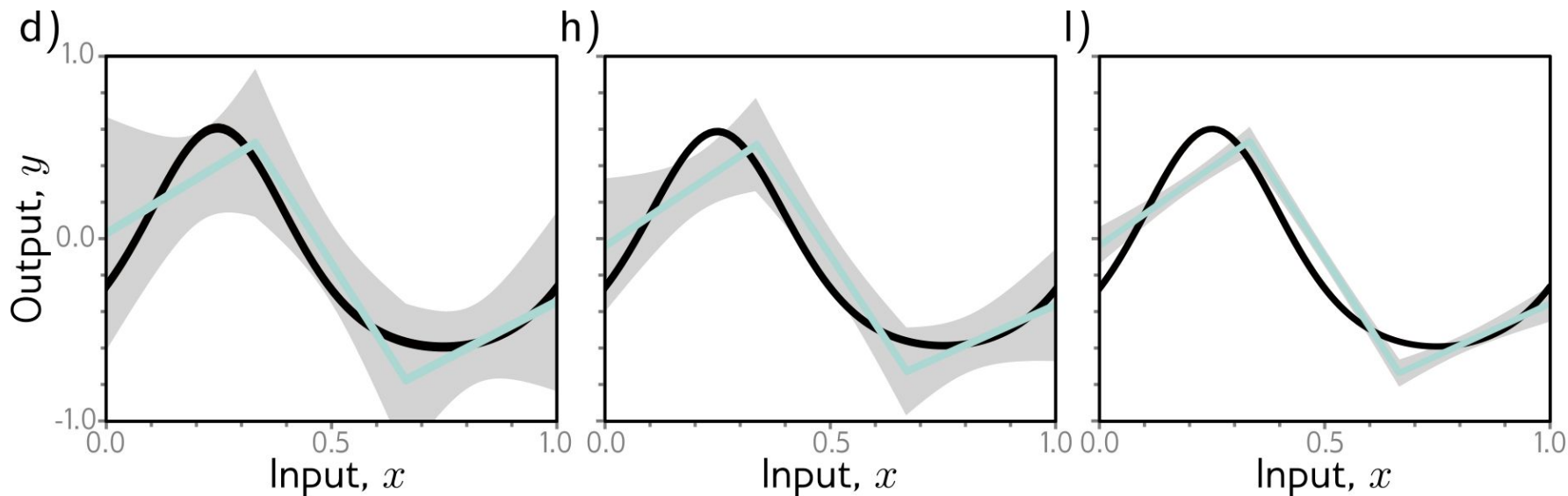
Reducing Errors : Noise

- There is nothing we can do to circumvent this, as it arises due to inherent nature of the data and the collection process.
- Some transformations can be used to make the 'signal' smoother but nothing can be done to remove it altogether.



Reducing Errors : Variance

- Just add more data!
- The image below shows variance in a dataset using 6, 10 and 100 samples, note how the variance decreases with more data.

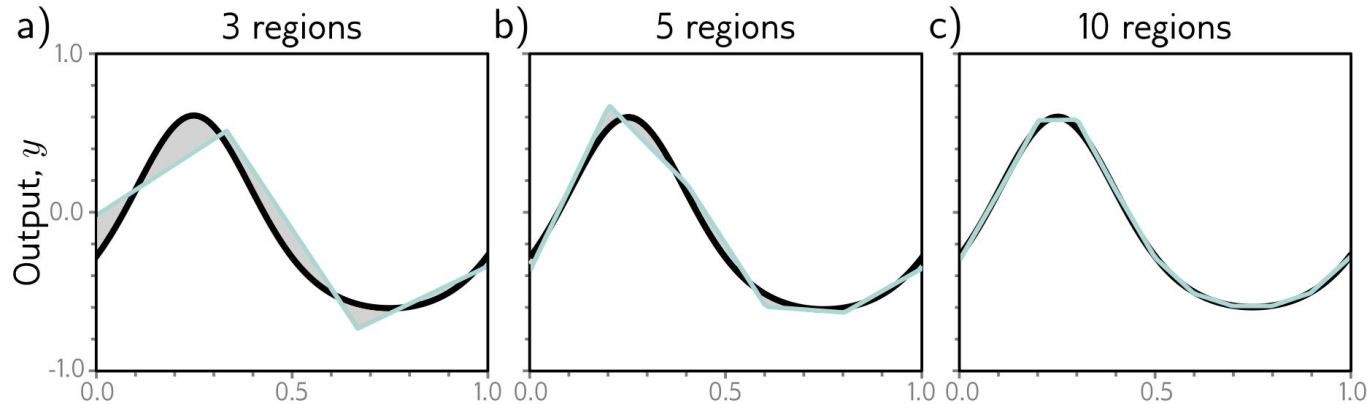


Reducing Errors : Bias

The bias term results from the inability of the model to describe the true underlying function.

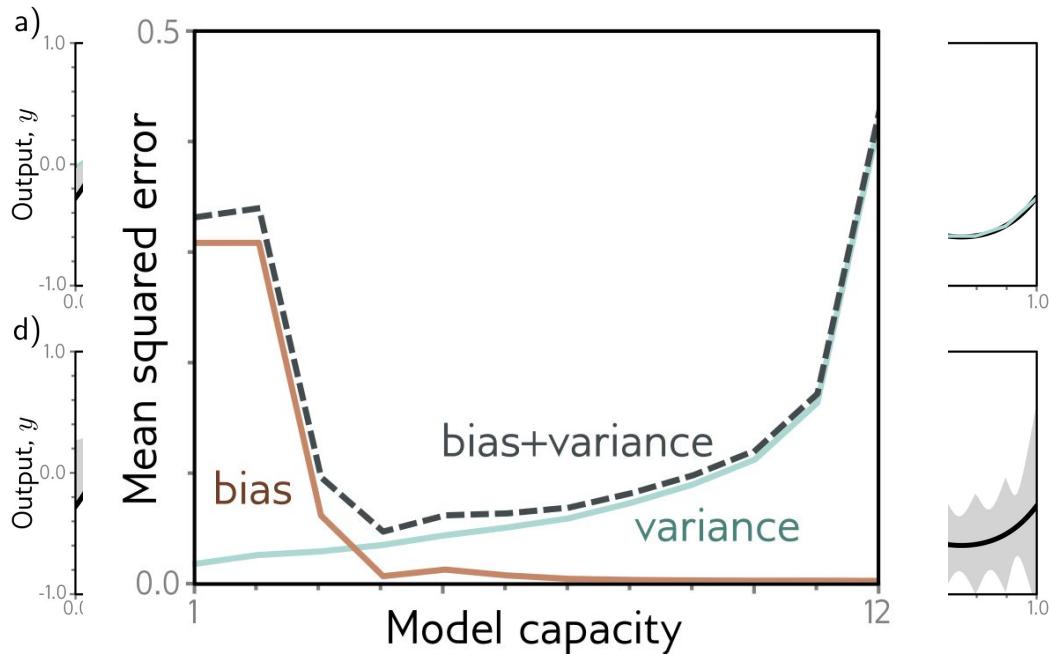
This suggests that we can reduce this error by making the model more robust by increasing its hidden units and/or hidden layers.

The image below showcases how adding new neurons to create more regions affected the models ability to replicate the original curve of the dataset.



Bias Variance Tradeoff

- For a **fixed-size training dataset**, the variance term typically increases as the model capacity increases.
- Consequently, increasing the model capacity does not necessarily reduce the test error.
- This is known as the bias-variance trade-off.



Choosing Hyperparameters

- What are Hyperparameters?
 - Structural: Number of hidden layers, units per layer (architecture).
 - Algorithmic: Learning rate, batch size, optimization type.
- The Evaluation Workflow
 - To ensure a model generalizes to new data, we split the data into three parts:
 - Training Set: Used to learn model parameters (weights).
 - Validation Set: Used to compare different hyperparameter combinations and select the best version.
 - Test Set: Used once at the very end to estimate true performance on unseen data.

Choosing Hyperparameters

- Challenges of Hyperparameter Search
 - Complex Space: Many choices are discrete (e.g., number of layers) or conditional (e.g., layer size depends on layer count).
 - No Gradients: Unlike weights, we cannot use gradient descent to optimize hyperparameters.
 - Computational Cost: Each "trial" requires training a full model from scratch.
 - Search Methods: Requires intelligent sampling (e.g., Random Search, Bayesian Optimization) rather than exhaustive testing.

Initialization Strategies

Method	Best Activation	Core Idea	Why use it?
Zero	None	All $W = 0$	Don't. It fails to break symmetry.
Random	Any (with luck)	Small random values	Simple, but risky for deep networks.
Xavier	Sigmoid / Tanh	Balances n_{in} and n_{out}	Prevents vanishing/exploding gradients in shallow/saturating nets.
Kaiming	ReLU / Leaky ReLU	Accounts for ReLU "off" state	Essential for deep networks using ReLU.